(Blind) Multimodal Bot Detection on X

Hamza Rashid
Project in Computer Science
McGill University

***Abstract*—This research investigates multimodal architectures for bot detection on X, in a *blind* prediction setting, where each account is observed exactly once at inference time, and graph information is not available. Our work begins with benchmarking conventional classifiers—a Random Forest on text, temporal, and topic distribution features, yielding moderate baseline performance. We then explore various multimodal approaches to combat manipulated features, with an emphasis on DNA-inspired behavior modeling to obtain dynamic user representations. For this, we embed the user description with TwHIN-BERT and model posting behavior with two DNA formulations, one that captures content across posts, and another capturing inter-post times. CNN embeddings of inter-post time DNA may capture latent temporal patterns that conventional methods miss. Three strategies for modality interaction are explored; a Gated Multimodal Unit, a cross-attention mechanism, and a sparsely gated mixture-of-experts (MoE) using self-attention to achieve modality interaction. The MoE yields our strongest detector, exhibiting rich representational capacity due to its conditioning on user sub-communities. Our findings show that a multimodal approach is robust against feature manipulation and exhibits strong cross-lingual transfer, and a divide-and-conquer architecture complements these effects by modeling the heterogeneous user communities on X.**

I. Introduction

Social media platforms like X (formerly Twitter) have enabled unprecedented global connectivity. As an open platform, X is vulnerable to malicious content, often originating from automated accounts—bots—that manipulate discourse, spread misinformation, or artificially amplify content. Detecting and mitigating such bot activities remains a challenging yet essential task for preserving authentic online interactions.

Most current work in bot detection is conducted in a fixed supervised setting where methods are optimized over known data. These datasets are often non-representative of the evolving behaviors exhibited by sophisticated bots. Consequently, there is a need for detection methods that generalize beyond known bot patterns.

We address this challenge by evaluating a range of bot detection strategies in a *blind* prediction setting, where each account is observed exactly once at inference time. This is achieved through a competition infrastructure where real tweet data automatically guide the generation of private bot data.

To detect these bots, we begin with classical machine learning techniques relying on extensive feature engineering. We employ a Random Forest classifier incorporating textual features, temporal posting patterns, and inferred topic distributions. While it achieves moderate baseline performance, its reliance on static derived features makes it gullible to manipulation strategies employed by nontrivial bots, such as copying tweets and descriptions from genuine users. This baseline reveals the extensive feature engineering required to keep up with evolving bots under classical approaches.

We thus explore various multimodal architectures to combat manipulated features, with an emphasis on DNA-inspired behavior modeling to obtain dynamic user representations. Our method combines user description embeddings with posting behavior encoded through two distinct DNA formulations: one capturing content across posts and another encoding inter-post timing patterns. In addition to the known efficacy of content DNA (Ilias et al., 2023), our experiments demonstrate that CNN (Convolutional Neural Network) encodings of inter-post time DNA may capture latent temporal patterns that conventional methods miss.

We explore several techniques for learning cross-modal interactions, including GMUs (Gated Multimodal Unit) (Arevalo et al., 2017), cross-attention mechanisms, and a sparsely gated MoE (Mixture-of-Experts) (Shazeer et al., 2017) using self-attention to achieve modality interaction. Our results highlight the MoE's superior representational capacity, effectively modeling the heterogeneous user communities on X. Our work emphasizes (1) the robustness of multimodal approaches and dynamic representations against feature manipulation, and (2) the potential of divide-and-conquer architectures in modeling the diverse user behaviors found on X.

II. Dataset and Experimental Setup

Our experiments are conducted within the "Bot Or Not" (B.O.N.) competition infrastructure, designed specifically for assessing bot detection strategies in realistic social media scenarios. This environment features competing deployments between bot and detector developers, simulating genuine activities on the X platform.

A. Data Characteristics

The dataset comprises real posts gathered from the X API across multiple collection sessions, each lasting the same duration but occurring on different days. For each session, posts are initially collected based on predefined sets of topical keywords. After gathering initial posts, users flagged as obvious bots are removed, and the dataset is expanded by including all additional posts from the remaining users.

Posts are text-only and only static metadata is kept; multimedia content and dynamic counts (likes, followers, etc.) are excluded. Hyperlinks are normalized to `https://t.co/twitter_link`, and user mentions are

anonymized to `@mention`. Accounts with 10–100 posts per session are kept to ensure sufficient data.

Two JSON files are produced per session: (i) **posts** (text, timestamp, post ID, author ID, language) and (ii) **users** (user ID, profile fields, tweet count, activity z-score).

B. Session Structure

Sessions are divided into multiple sequential sub-sessions to mimic the real-time flow of social media feeds. Bots generate content incrementally across these sub-sessions, with the requirement to post 10–100 posts by session completion. Detector deployments subsequently analyze the fully compiled datasets post-session to identify bots, providing classification along with the bot-confidence score for each account.

C. Technical Setup

Participant submissions (both bot and detector code) are managed via public GitHub repositories, cloned into Docker images, and deployed on EC2 instances. Submissions are required to execute fully within a one-hour timeframe, after which automated shutdown procedures terminate any ongoing processes.

III. Methodology

We aim to model user behavior from a variety of perspectives, since nontrivial bots engage in some form of feature manipulation (e.g., tweet content, posting times, or interactions such as retweets, likes, and follower-following relationships).

Our study proceeds in two stages:

1) **Feature-engineered baseline.** A Random Forest (RF) trained on text embeddings, topic histograms, and temporal statistics.
2) **Multimodal neural models.** Architectures that fuse user description embeddings with *digital DNA* images capturing content, timing, and (optionally) emoji patterns.

Complete results are shown in Table I.

A. *Baseline - Random Forest*

We employ a Random Forest for its flexibility with data types, allowing rapid feature exploration, and proven efficacy in bot detection, most notably in Botometer (Davis et al., 2016).

The average human-to-bot ratio across the sessions is roughly $14:1$, making the datasets moderately imbalanced. Thus, the Random Forest uses class weights inversely proportional to class frequency to improve bot recall. It consists of 100 trees, and is trained and evaluated using stratified 80–20 train–test splits.

1) ***Session 11*** *– Embeddings + LDA:* For Session 11, we extract document-level tweet embeddings for each user with all-MiniLM-L6-v2 (Reimers and Gurevych, 2021) to capture textual behavior. Further, we use LDA (Blei et al., 2003) topic modeling under the assumption that bots target a few topics disproportionately, resulting in vectors with unusually large values in a few entries. We perform minor text preprocessing by converting URLs, mentions, and hashtags into special tokens (`<URLURL>`, `<UsernameMention>`, `<HashtagMention>`) to remove noise from LDA, which relies on term frequencies. LDA is performed with stopword exclusion performed internally, leaving the contextual tweet embeddings unaffected.

Result: The model exhibits low confidence in the bot class, labeling every account as human. This likely owes to LDA's reliance on predefined topics from training data, which vary across sessions and thus produce sparse vectors on unseen data.

2) ***Session 12*** *– Mean Embedding + BERTopic + Temporal Statistics:* In this session, we shift to using per-tweet embeddings and adopt BERTopic (Grootendorst, 2022) for more dynamic topic modeling. Additional temporal statistics are introduced to capture behavioral rhythms. To counteract low confidence scores in the bot class, we perform min-max scaling of the confidences after inference.

Implementation Summary: Individual tweets are embedded and clustered into topics using BERTopic (HDBSCAN (McInnes et al., 2017)) to generate a normalized topic histogram vector for users. Then, a mean-pooling of tweet-level embeddings provides the final text encodings. Finally, the mean and standard deviation of raw timestamps are computed for each user.

Result: The model achieves moderate results in this session, improving in both recall and precision. This likely owes to (1) scaled confidence scores, (2) time-based features providing obvious signals and (3) batch-independent topic modeling providing informative features to the classifier. The time-based features likely provide the most utility, since many bots at this point do not modify their posting times.

3) ***Session 13*** *– Enhanced Temporal Behavior Modeling:* Session 13's approach is summarized as follows: We compute the mean and standard deviation of time gaps between consecutive posts instead of raw timestamps. Then, raw timestamps are ordered chronologically and split into 10 disjoint intervals, and duplicate timestamps are computed per interval to produce a ten-dimensional vector of these counts. Finally, these features are added to those from the previous session.

Result: This is the strongest-performing baseline thus far. The timestamp partitioning captures bursty behavior, and capturing the increased time gap variance helps account for bots using a randomized post scheduler. However, similar to the previous session, there are many bots at this point that do not modify their posting times, skewing the utility of timestamp partitioning.

4) ***Session 14*** *– Topic Statistics + Temporal Statistics + Intra-Topic Similarity:* Hashtags are partially preserved (`#arena` $\rightarrow$ `<HashtagMention:arena>`) to keep topic cues often used by bots. Tweet embeddings are normalized to prevent variance from dominating downstream signals. Then, the topic histogram is replaced with topic mean, variance, and standard deviation in order to remove noise. Finally, the mean, median, and standard deviation of pairwise cosine similarities across tweet embeddings are computed for each user (averaged over their topics).

Result: By Session 14, the bots evolve. That is, under the same training conditions, this model outperforms its predecessor on Session 13 data, but the results of 14 decline. This indicates the failure of a feature engineering approach to adapt to evolving bots.

B. Summary and Reflection on Random Forest Baseline

Our progression through four sessions using Random Forest classifiers illustrates the evolving challenge of distinguishing bots from humans on X through feature engineering alone.

While the Random Forest baseline allows us to test many ideas rapidly, an increasing complexity in feature engineering is needed to keep up with evolving bots. They copy tweets and descriptions from genuine users, and eventually employ LLMs fine-tuned to generate natural-sounding text. These bot strategies reduce informative signals in our features. Further, our model relies on many static features (e.g., mean and standard deviation of time between posts), which are also vulnerable to manipulation (e.g., a sophisticated post scheduler mimicking human patterns).

C. Multimodal Architectures

Recognizing the constraints of feature engineering, we implement multimodal neural architectures inspired by recent literature *"Multimodal Detection of Bots on X (Twitter) using Transformers"* (Ilias et al., 2023), and *"BotMoE: Twitter Bot Detection with Community-Aware Mixtures of Modal-Specific Experts"* (Liu et al., 2023).

We explore the following designs for modality interaction (text and DNA):

- GMU: Uses a gating mechanism to learn modality interactions.
- Cross-Attention (CA): Uses multihead attention to learn modality interactions.
- MoE: Dedicates a MoE model for each modality, subsets of the MoE specialize in different user types, and self-attention is used to achieve cross-modal interactions.

These architectures not only provide greater modeling capacity but also offer more robustness against feature manipulation, as they do not rely on manually crafted statistics. Instead, they operate directly over sequences of observed user behavior and learn to extract structure.

*1) **DNA Embedding Extraction**:* To model user behavior beyond text embeddings, we construct symbolic sequences known as digital DNA, across the user's timeline chronologically, representing content, timing, and emoji usage across tweets. These sequences are then converted into images and embedded via a CNN encoder:

1) Each symbol is mapped to an 8-bit grayscale intensity value (e.g., $\texttt{N} \mapsto 0$, $\texttt{U} \mapsto 64$, ..., $\texttt{X} \mapsto 255$).
2) The sequence of grayscale values is reshaped into a square matrix: if the sequence has length L, the smallest $n \in \mathbb{N}$ such that $n^2 \geq L$ is selected. The sequence is zero-padded to length n^2 and reshaped to $(n \times n)$.
3) This grayscale image is resized depending on the CNN encoder, using nearest-neighbor interpolation.
4) The image is then converted to RGB format by replicating the grayscale channel.
5) The RGB tensor is normalized using ImageNet statistics and passed through a pretrained CNN encoder.
6) The embedding is obtained by either (1) projecting a mean-pooling of the last hidden layer, or (2) directly extracting the last hidden layer.

Content DNA. Each tweet is mapped to a symbol based on the type of entities it contains:

- `N` — No entities (plain text).
- `U` — Contains a URL.
- `H` — Contains one or more hashtags.
- `M` — Contains one or more mentions.
- `X` — Contains a combination of multiple entity types.

Time DNA. Assigns symbols based on the time elapsed between consecutive tweets:

- `n` — User's first post (no delta).
- `o` — $<$ 1 second since the user's previous post.
- `s` — $\geq$ 1 second and $<$ 1 minute.
- `m` — $\geq$ 1 minute and $<$ 1 hour.
- `h` — $\geq$ 1 hour and $<$ 6 hours.
- `u` — $\geq$ 6 hours and $<$ 12 hours.
- `x` — $\geq$ 12 hours.

Emoji DNA. We categorize emojis (Unicode Consortium, 2020) in tweets using a precompiled lookup table. Each tweet is assigned a category based on its emojis:

- `S` — Smileys & Emotion.
- `P` — People & Body.
- `A` — Animals & Nature.
- `F` — Food & Drink.
- `T` — Travel & Places.
- `R` — Activities.
- `O` — Objects.
- `Y` — Symbols.
- `L` — Flags.
- `N` — No emoji present.
- `X` — Multiple emoji categories.

In all cases, these DNA images serve as behavioral *fingerprints*, allowing CNNs to learn latent signals over them.

*2) **Gated Multimodal Unit (GMU)**:* The GMU allows the network to dynamically control how much each modality contributes to the final hidden representation.

In the bimodal setting, where we consider two input modalities $\mathbf{x}_v$ and $\mathbf{x}_t$ (e.g., DNA and text), the equations governing the GMU are as follows:

$$\mathbf{h}_v = \tanh(\mathbf{W}_v \cdot \mathbf{x}_v) \tag{1}$$

$$\mathbf{h}_t = \tanh(\mathbf{W}_t \cdot \mathbf{x}_t) \tag{2}$$

$$\mathbf{z} = \sigma(\mathbf{W}_z \cdot [\mathbf{x}_v, \mathbf{x}_t]) \tag{3}$$

$$\mathbf{h} = \mathbf{z} * \mathbf{h}_v + (1 - \mathbf{z}) * \mathbf{h}_t \tag{4}$$

$$\mathbf{\Theta} = \{\mathbf{W}_v, \mathbf{W}_t, \mathbf{W}_z\} \tag{5}$$

Here, $\mathbf{h}_v$ and $\mathbf{h}_t$ are hidden activations learned on modalities $\mathbf{x}_v$ and $\mathbf{x}_t$, $[\cdot, \cdot]$ denotes concatenation, and $\mathbf{\Theta}$ denotes

learnable parameters. Cross-modal interactions are learned in the parameters $\mathbf{W}_z$, the weighting mechanism $\mathbf{z}$ then determines how much each modality contributes to the fused representation $\mathbf{h}$. In practice, implementations tend to depart[1] from some of the original equations, including ours[2].

Resembling the work of Ilias et al. (2023), our initial efforts with the GMU used the following modalities:

- Text: User description embedding from TwHIN-BERT (Zhang et al., 2022).
- DNA: Content-DNA embedding from MobileNetV2 (Sandler et al., 2018).

The first iteration consists of a two-layer classification head applied to the GMU's output. This model labels every account as human, and several adjustments are due. Subsequent attempts introduce refinements such as: two-layer modality-specific projections, layer-normalization, global residual connections, and modality-specific dropout.

*a) **Incorporating Time-DNA**:* While refining our GMUs, we incorporate time-DNA embeddings as a third sub-modality. This addition improves performance, supporting our intuition that dynamic temporal representations can capture richer patterns than static temporal features.

While the GMU refinements yield nontrivial improvements, development requires numerous adjustments to stabilize training and optimize performance. This raises concerns about generalizability; can such a carefully balanced architecture handle different modalities or scale to more modalities?

We thus employ cross-attention for learning modality interactions, expecting that our model can achieve nontrivial results with fewer architectural tweaks.

*3) **Cross-Attention (CA)**:* We first learn a single representation for the DNA modality (content and time) to later apply cross-attention with the text modality (user description). We find that the GMU learns the DNA representation effectively, despite its intended multimodal use. Our cross-attention method is illustrated in Figure 1.

To achieve modality interaction, we employ the multihead-attention approach from Ilias et al. (2023), which computes scaled dot-product attention (Vaswani et al., 2017)

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V \tag{6}$$

in one layer where the text modality corresponds to the query (Q) and DNA corresponds to the key (K) and value (V), and another layer in which the modalities swap roles. We then average-pool the attention outputs to form a final fused representation.

With the fused representation, we begin with a two-layer classification head and iteratively improve our architecture. This required fewer refinements than our GMU approach.

[1]Demonstration https://github.com/johnarevalo/gmu-mmimdb/blob/master/model.py by Arevalo et al. (2017) omits modality projections and applies nonlinearity directly to the inputs.

[2]Concatenates the hidden activations.

*a) **Sessions 16–17** – Description, Content, Time:* For Sessions 16 and 17, we employ the cross-attention approach, and train the model with class-weighted cross-entropy loss to improve bot recall. The model's design follows:

- Pre-Layer normalization (Xiong et al., 2020) and residual connections are applied at each attention stage.
- The cross-attention outputs are concatenated into a sequence and fed through a single position-wise FFN.
- The FFN uses pre-layer normalization, and a residual connection is added to the FFN outputs.
- The FFN outputs are mean-pooled, and a two-layer classification head generates the final logits for bot detection.

Result: The model underperforms relative to previous sessions. Given the evolving bots and lack of hyperparameter tuning at this stage, such results are expected. However, the model shows strong cross-lingual transfer, achieving the highest recall in Session 17, the first French dataset, despite training only on English data up to this point. This result demonstrates the importance of a multimodal approach.

*b) **Sessions 18–19** – Description, Content, Time, Emoji:* Noticing that LLM-powered bots exhibit obvious tells in their emoji usage across posts (e.g., repeatedly using a single category), we extend the model's GMU to fuse three DNA embeddings for Sessions 18 and 19: content, time, and emoji. The model is trained with focal loss (Lin et al., 2017), proving to be more effective than weighted cross-entropy for dealing with the class imbalance.

Result: Bot recall does not improve on the previous attempt. However, precision increases three-fold and F1 increases two-fold in both languages since we had more time for hyperparameter tuning. The sparse nature of the emoji embeddings (not every account uses emojis) likely adds noise to the model, which would explain the increased sensitivity to hyperparameter configurations. We therefore remove the emoji-DNA from subsequent approaches. Future refinements may involve curating a more appropriate categorization system for emojis.

c) Conclusion: The cross-attention approach shows promising results, with the highest recall in Session 17, but due to the tuning requirements and use of emoji-DNA, it underperforms in other sessions.

Moreover, it is computationally expensive to scale the cross-attention approach beyond two modalities. For example, three modalities would require six multihead-attention layers to consider each bimodal interaction in both directions.

A natural alternative is to employ a single self-attention layer where each token corresponds to a modality. Further, the user diversity on X poses a challenge in learning modality representations to provide to the fusion logic. In the case of the self-attention layer, we find that MoEs learn these representations effectively.

*4) **Mixture-of-Experts (MoE)**:* To address the challenges of (1) model scalability and (2) diverse user behavior on X, we employ the sparsely gated MoE (Shazeer et al., 2017) strategy used by BotMoE (Liu et al., 2023). This approach leverages specialized experts conditioned on (learned) user

sub-communities, enabling community-aware representation learning.

a) Expert Assignment and Gating: For each modality, user embeddings are fed into a **gating network**:

$$G(\mathbf{x}) = \text{softmax}(\text{KeepTopK}(W_g\mathbf{x}, k)) \tag{7}$$

where W_g are learnable weights, $\mathbf{x}$ is the input embedding, and `KeepTopK` retains the top k highest gating scores for sparse expert activation.

b) Expert Aggregation: The final representation for modality $m \in \{t = \text{text}, d = \text{DNA}\}$ is a weighted sum of expert outputs:

$$\mathbf{z}_i^{(m)} = \sum_{j=1}^{n} G(\mathbf{x}_i^{(m)})_j E_j(\mathbf{x}_i^{(m)}) \tag{8}$$

where $G(\mathbf{x}_i^{(m)})_j$ is the probability that user i belongs to community j, and $E_j(\mathbf{x}_i^{(m)})$ denotes the j^{th} expert's output (a two-layer MLP). The final value $\mathbf{z}_i^{(m)}$ is a weighted sum of experts selected by the gating network.

c) Modality Interaction and Consistency: After expert aggregation, representations across modalities interact via multihead **self-attention**:

$$\{\tilde{\mathbf{z}}^{(t)}, \tilde{\mathbf{z}}^{(d)}\} = \text{SelfAttn}(\{\mathbf{z}^{(t)}, \mathbf{z}^{(d)}\}). \tag{9}$$

Additionally, we perform the cross-modal consistency evaluation from Liu et al. (2023), aiming to capture feature manipulation.

$$\mathbf{z}^{(con)} = \text{flatten}(\text{Conv2D}(\text{sample}(\mathbf{M}))). \tag{10}$$

This is done by applying a 2D convolution[3] on a sample (obtained via fixed-pooling) of the attention matrix $\mathbf{M}$.

d) Classification Head: The attention outputs and consistency vector are concatenated and passed through a final classifier:

$$y_i = W_o \cdot [\tilde{\mathbf{z}}^{(t)}, \tilde{\mathbf{z}}^{(d)}, \mathbf{z}^{(con)}] + b_o \tag{11}$$

where W_o and b_o are learnable parameters.

e) Loss Function: We optimize a cross-entropy loss with L2-regularization to mitigate overfitting, and the **balancing loss** from Shazeer et al. (2017) to encourage balanced expert usage:

$$\mathcal{L} = -\sum_{i\in\mathcal{Y}} t_i \log y_i + \lambda_1 \sum_{w\in\theta} w^2 + \lambda_2 \sum_{i\in\mathcal{Y}} \sum_{m\in\{t,d\}} BL(\mathbf{x}_i^{(m)}) \tag{12}$$

where t_i is user i's true label, θ denotes model parameters (L2-regularization), λ_1 and λ_2 are hyperparameters weighing their corresponding terms, and $BL(\cdot)$ is the balancing loss from Shazeer et al. (2017):

$$BL(\mathbf{x}) = w_{imp} \cdot CV(G(\mathbf{x}))^2 + w_{ld} \cdot CV(P(\mathbf{x}, i))^2. \tag{13}$$

Here, CV is the coefficient of variation, $P(\mathbf{x}, i)$ is the smooth function defined in Shazeer et al. (2017), and w_{imp} and w_{ld} are hyperparameters weighing expert importance and load.

[3]Our implementation (Sessions 20-21), similar to BotMoE https://github.com/lyh6560new/BotMoE, uses the flattened fixed-pooling output directly.

*f) **Sessions 20–21** – Text, DNA:* For Sessions 20 and 21, we employ the outlined MoE approach (illustrated in Figure 2) with the following modalities:

- Text: User description + Five tweets sampled evenly across their timeline (partitioning their posts into equal chunks). Both are encoded with TwHIN-BERT, with a mean-pooling applied to tweets.
- DNA: Content + Time, encoded with MobileViT-small (Mehta and Rastegari, 2021).

Each modality-specific MoE contains two experts, and the gating network selects the top expert (i.e., $k = 1$ in (7)).

Result: The MoE yields our strongest detector, achieving perfect precision and the highest recall in Session 20 (English), and the highest recall in all of the French Sessions. We reason that the model does not require treatment of class imbalance because the reduced per-expert user-diversity makes it easier for experts to learn informative representations.

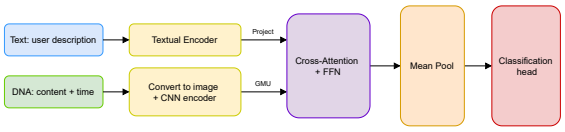

Fig. 1: Text-DNA Cross-Attention bot detection framework.

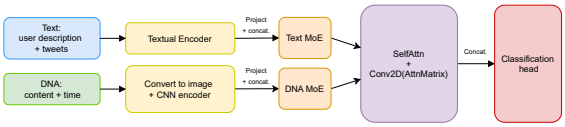

Fig. 2: Text-DNA MoE bot detection framework.

IV. Discussion

In this section we interpret the results in Table I and discuss how they illuminate the challenges of detecting increasingly sophisticated bots on X. We frame the discussion around three axes that emerged as decisive throughout the competition lifecycle: (i) the tension between bots' **manipulated static features** and detectors' **dynamic behavior-centric features**, (ii) the **diversity of user communities**, and (iii) the **scalability** of detection architectures.

A. *Key empirical trends*

The baseline Random Forests (RF) established in Sessions 11–14 offer an instructive point of comparison: they achieved $\geq 95\%$ overall accuracy in every session, yet bot-recall vacillated from 0.00 (Session 11) to 0.905 (Session 13), before sliding back to 0.690 (Session 14). With a human-to-bot ratio of roughly 14 : 1, these swings illustrate a classic imbalance pitfall: accuracy is dominated by the majority class, while static, hand-engineered features fail to adapt to evolving bots (the final RF outperforms its predecessor, but performance declines in Session 14).

Transitioning to the *Cross-Attention* (CA) model in Sessions 16–19 reverses this failure mode: recall stabilizes between 52% and 72%, but precision remains modest (14–62%), despite the addition of noisy features (emoji-DNA). This shift underscores the benefit of the multimodal approach (adaptability reflected in metric stability and cross-lingual transfer) but also reveals the difficulty of learning rich cross-modal interactions with evolving bots and imbalanced labels.

The MoE architecture deployed in Sessions 20–21 marks a decisive improvement. Perfect precision (100%) and a recall of 67.7% in Session 20 demonstrate that a multimodal approach is complemented by a divide-and-conquer architecture that attends to the heterogeneous communities on X. The results also show that community-aware specialization can reconcile the precision-recall trade-off without explicit re-balancing of the loss. Further, the performance uplift is achieved with fewer parameters than the cross-attention approach, suggesting that a small divide-and-conquer architecture can outperform a large monolithic model in bot detection.

B. *Manipulated* vs. *dynamic features*

Early sessions revealed that bots actively engaged in feature manipulation strategies, such as copying profile descriptions and tweet content from genuine users. Consequently, feature sets anchored to surface-level cues suffered from feature dilution. Digital DNA representations address this weakness by implicitly encoding temporal dynamics that are harder to spoof (but still vulnerable). CNN embeddings of DNA images capture higher-order patterns (e.g., periodic behavior) beyond the reach of coarse summary statistics. The progressive gain from Session 12 to 14 (RF models) already hinted at the importance of dynamic signals; the multimodal results confirm that learning these patterns directly, rather than engineering them, yields more robust detectors.

C. *User diversity and community-aware modeling*

Bots on X do not form a homogeneous cluster; they range from single-issue spam accounts to multilingual LLM-powered impersonators. Treating all users the same can therefore lead to a weak detector. The MoE's sparsely gated experts act as implicit community detectors, allowing the model to attend to behaviorally coherent sub-populations (e.g., language communities, topical niches). The cross-lingual recall gains observed in French (Session 21) and the balanced expert utilisation encouraged by the balancing loss are practical evidence of this effect.

V. Limitations

In addition to the lack of graph information and dynamic user metadata, the experiments presented in this study must be interpreted in light of several methodological limitations.

A. Dataset Bias and Temporal Scope

Session datasets tend to cover **only two days** of posts, and initial tweet harvests are driven by *pre-defined topical keywords*. Consequently, the data are *not* a random or longitudinal sample of the X platform. Three practical implications follow:

Session	Model	Total Users	Acc.	Prec.	Rec.	F1
11 (En)	RF	420	0.9929	0.0000	0.0000	0.0000
12 (En)	RF	393	0.9669	0.7333	0.5500	0.6286
13 (En)	RF	394	0.9949	1.0000	0.9048	0.9500
14 (En)	RF	402	0.9577	0.7143	0.6897	0.7018
15 (En)	N/A	–	–	–	–	–
16 (En)	CA	438	0.8151	0.1429	0.5714	0.2286
17 (Fr)	CA	306	0.8105	0.1970	0.7222	0.3095
18 (En)	CA	455	0.9275	0.5714	0.5263	0.5479
19 (Fr)	CA	288	0.9132	0.6176	0.6364	0.6269
20 (En)	MoE	448	0.9777	1.0000	0.6774	0.8077
21 (Fr)	MoE	283	0.9682	0.8800	0.7857	0.8302

TABLE I: Performance of our bot detectors across sessions. Precision, recall, and F1 are computed for the bot class. Session language is either English (En) or French (Fr).

1) **Temporal representativeness.** Results obtained with dynamic features (e.g., content- and especially time-DNA) are biased towards a short timeframe.
2) **Topic coverage.** Keyword seeding biases the corpus towards the chosen themes of each session. Bots—or humans—operating outside those topical clusters are under-represented, limiting external validity.
3) **Active-user bias.** We retain only accounts that issue *at least 10 posts* over the two days of collection. Since sequence length is a bottleneck for the DNA modality, the obtained results are likely optimistic.

B. ChatGPT bias and Annotation bottleneck

Most bot developers in the B.O.N. framework relied on **ChatGPT-generated content** to power their accounts. The detectors may implicitly learn stylistic artefacts specific to ChatGPT, limiting the generalizability of results and jeopardizing performance against future bots using different strategies.

Annotation bottleneck and latent bots. Running sessions requires annotating hundreds of accounts. If annotation relies solely on simple automated methods, sophisticated bots might be mislabeled as humans, and genuine users might be mislabeled as bots, all before the bot developer code is deployed. The potential mislabeling further limits the generalizability of the obtained results.

VI. Conclusion

This study set out to detect social bots on X in a *blind* prediction setting, where each account is observed exactly once at inference time and no graph information is available. We began with classical Random Forest baselines that relied on hand-engineered, largely static features. While these models occasionally achieved notable results, they failed to adapt to evolving bots that engaged in feature manipulation and employed LLMs fine-tuned to generate natural-sounding text.

Our multimodal exploration revealed notable strategies. First, behavior-centric representations such as content- and time-DNA capture longitudinal patterns—burstiness, scheduling regularities, and entity-mixing—that are tedious for bots to imitate consistently, thereby hardening the detector against superficial mimicry. Second, the sparsely gated,

community-aware MoE complements a multimodal approach: by routing behaviorally similar users to the same specialist, each expert faces reduced variability and can learn sharper representations, yielding a stronger detector.

Taken together, these findings suggest two design strategies for blind bot detection: (i) Employ a multimodal approach, embracing dynamic, longitudinal signals. (ii) Allocate model capacity adaptively—specialized experts for heterogeneous user sub-populations.

Future Work. Grounded in these findings and the dataset limitations detailed in Section V, we outline three possible next steps:

1) *DNA-driven clustering and weak supervision*. The CNN embeddings (and even raw sequences) already produced for content- and time-DNA can seed unsupervised clustering. Grouping accounts by DNA similarity might (i) expose emergent bot communities and (ii) act as a lightweight pre-filter that forwards only suspicious clusters to heavier detectors.
2) *Mitigating session bias with broader windows and stratified sampling*. To reduce the two-day temporal scope and keyword-driven topical skew, future sessions should (a) stagger collection windows over several weeks, (b) remove topic seeding or broaden it, and (c) deliberately include low-activity accounts below the current 10-post threshold (and allow the bots to have low activity).
3) *Refining dataset curation to surface "wild" bots*. The present B.O.N. pipeline filters out non-developer bots up-front, yet real platforms teem with legacy or clandestine bots. A more nuanced filtering workflow—combining heuristic rules, shallow ML screens, and manual spot checks—should flag these latent bots *for inclusion*, yielding a benchmark that better reflects on-platform diversity and challenges future detectors.

Overall, multimodal divide-and-conquer detectors that embrace behavioral dynamics offer a promising defense against evolving social bots. The MoE framework demonstrates that such models can be simultaneously effective and computationally efficient, which are desirable properties for real-world applications in bot detection.

Acknowledgment

The author would like to thank Dr. Derek Ruths for his guidance and for permitting the research to focus on model architectures.

The author also thanks Émilie Ducharme for maintaining and updating the competition infrastructure.

The author further thanks Shangbin Feng (PhD student, University of Washington) for sharing the TwiBot-22 dataset (Feng et al., 2022), which was used for development purposes.

The conclusions and recommendations expressed in this material are those of the author and do not necessarily reflect the views of any affiliated institutions or individuals.

Code Availability

The implementation code is available at the following repository: https://github.com/HzaRashid/BattleBotsDetector

References

John Edison Arevalo, Thamar Solorio, Manuel Montes-y G'omez, and Fabio A. Gonz'alez. Gated multimodal units for information fusion. *CoRR*, abs/1702.01992, 2017. URL https://arxiv.org/abs/1702.01992.

David M. Blei, Andrew Y. Ng, and Michael I. Jordan. Latent dirichlet allocation. *Journal of Machine Learning Research*, 3:993–1022, 2003.

Clayton A. Davis, Onur Varol, Emilio Ferrara, Alessandro Flammini, and Filippo Menczer. Botornot: A system to evaluate social bots. In *Proceedings of the 25th International Conference Companion on World Wide Web*, pages 273–274. ACM, 2016.

Shangbin Feng, Zhaoxuan Tan, Herun Wan, Ningnan Wang, Zilong Chen, Binchi Zhang, Qinghua Zheng, Wenqian Zhang, Zhenyu Lei, Shujie Yang, Xinshun Feng, Qingyue Zhang, Hongrui Wang, Yuhan Liu, Yuyang Bai, Heng Wang, Zijian Cai, Yanbo Wang, Lijing Zheng, Zihan Ma, Jundong Li, and Minnan Luo. Twibot-22: Towards graph-based twitter bot detection. *arXiv preprint arXiv:2206.04564*, 2022. URL https://arxiv.org/abs/2206.04564.

Maarten Grootendorst. Bertopic: Neural topic modeling with a class-based tf-idf procedure. *arXiv preprint arXiv:2203.05794*, 2022. URL https://arxiv.org/abs/2203.05794.

Loukas Ilias, Ioannis Michail Kazelidis, and Dimitris Th. Askounis. Multimodal detection of bots on x (twitter) using transformers. *arXiv preprint arXiv:2308.14484*, 2023. URL https://arxiv.org/abs/2308.14484.

Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 2980–2988. IEEE, 2017. doi: 10.1109/ICCV.2017.324. URL https://arxiv.org/abs/1708.02002.

Yuhan Liu, Zhaoxuan Tan, Heng Wang, Shangbin Feng, Qinghua Zheng, and Minnan Luo. Botmoe: Twitter bot detection with community-aware mixtures of modal-specific experts. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR '23, pages 485–495, Taipei, Taiwan, July 2023. ACM. doi: 10.1145/3539618.3591646.

Leland McInnes, John Healy, and Steve Astels. hdbscan: Hierarchical density based clustering. *The Journal of Open Source Software*, 2(11):205, 2017. doi: 10.21105/joss.00205.

Sachin Mehta and Mohammad Rastegari. Mobilevit: Lightweight, general-purpose, and mobile-friendly vision transformer. *arXiv preprint arXiv:2110.02178*, 2021. URL https://arxiv.org/abs/2110.02178.

Nils Reimers and Iryna Gurevych. Making monolingual sentence embeddings multilingual using knowledge distillation. https://www.sbert.net, 2021. Accessed: 2025-04-25.

Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4510–4520, 2018. doi: 10.1109/CVPR.2018.00474. URL https://arxiv.org/abs/1801.04381.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc V. Le, Geoffrey E. Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *CoRR*, abs/1701.06538, 2017. URL http://arxiv.org/abs/1701.06538.

Unicode Consortium. Emoji ordering v13.0, 2020. URL https://www.unicode.org/emoji/charts-13.0/emoji-ordering.html. Accessed: 2025-04-30.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, pages 5998–6008. Curran Associates, Inc., 2017. URL https://papers.nips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.

Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. On layer normalization in the transformer architecture. *arXiv preprint arXiv:2002.04745*, 2020. URL https://arxiv.org/abs/2002.04745.

Haoyi Zhang, Yuhan Liu, Qinghua Zheng, and Minnan Luo. Twhin-bert: A robust twitter user representation model for social bot detection. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2879–2893. Association for Computational Linguistics, 2022. doi: 10.18653/v1/2022.emnlp-main.210. URL https://aclanthology.org/2022.emnlp-main.210.